This document will guide you to deploy your applications, frontend and backend to a Linux server, specifically to Ubuntu OS. Some details may differ from your project, please be cautious.

Context s

- In this deployment, all services will be run on the same Ubuntu server.
- The front end (React) will run from root (/) path.
- The backend (Next) will run from api (/api) path.

Updating the system

1. Run apt update

```
azureuser@my-server-01:~$ sudo apt update
```

2. Run apt upgrade

```
azureuser@my-server-01:~$ sudo apt upgrade
```

Installing Docker

1. Grab the docker installation script from MS Team.
2. Upload it to your Vm
3. Change mode by adding "x" for the owner.

```
azureuser@my-server-01:~$ chmod u+x docker_install.bash
```

4. Run the script

```
azureuser@my-server-01:~$ ./docker_install.bash
```

5. Answer "Y" for any confirmation.
6. Check if docker installed successfully.

```
azureuser@my-server-01:~$ sudo docker --version
Docker version 29.1.3, build f52814d
azureuser@my-server-01:~$
```

Prepare your project – The code

1. Grab Dockerfile for "react project" from MS Team.
2. Grab nginx.conf for "react project" from MS Team.
3. Add them to root directory of your React project (frontend).
4. Update your repository (push)
5. Grab Dockerfile for "Next project" from MS Team.
6. Add into your Next project.x
7. Update your repository.

Prepare your project – The remote server

1. Create a directory to store docker configurations and your projects.
   My common directory name is “dockerfiles”.

```
azureuser@my-server-01:~$ mkdir dockerfiles
azureuser@my-server-01:~$ ls -l
total 8
-rwxrw-r-- 1 azureuser azureuser  808 Jan  2 16:02 docker_install.bash
drwxrwxr-x 2 azureuser azureuser 4096 Jan  2 16:08 dockerfiles
azureuser@my-server-01:~$
```

2. Change to the directory.

```
azureuser@my-server-01:~$ cd dockerfiles/
azureuser@my-server-01:~/dockerfiles$
```

3. Clone your frontend / backend projects

```
azureuser@my-server-01:~/dockerfiles$ git clone https://github.com/pop2518hotmailcom/my-next-backend.git
Cloning into 'my-next-backend'...
Username for 'https://github.com': pop2518hotmailcom
Password for 'https://pop2518hotmailcom@github.com':
remote: Enumerating objects: 37, done.
remote: Counting objects: 100% (37/37), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 37 (delta 6), reused 37 (delta 6), pack-reused 0 (from 0)
Receiving objects: 100% (37/37), 63.79 KiB | 920.00 KiB/s, done.
Resolving deltas: 100% (6/6), done.
azureuser@my-server-01:~/dockerfiles$
```

```
azureuser@my-server-01:~/dockerfiles$ git clone https://github.com/pop2518hotmailcom/my-react-frontend.git
Cloning into 'my-react-frontend'...
remote: Enumerating objects: 27, done.
remote: Counting objects: 100% (27/27), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 27 (delta 4), reused 25 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (27/27), 255.18 KiB | 42.53 MiB/s, done.
Resolving deltas: 100% (4/4), done.
azureuser@my-server-01:~/dockerfiles$
```

   Change the Github URL to your Github repository URLs.

4. Add “.env” file into your front end project as shown below with your server’s DNS.

   VITE_API_URL=https://athiphathrn-server-01.southeastasia.cloudapp.azure.com/api

Create Docker Image

1. Run docker command to build image for each project, let's start with backend
    1. Change to the backend project directory

```
azureuser@my-server-01:~/dockerfiles$ cd my-next-backend-02/
azureuser@my-server-01:~/dockerfiles/my-next-backend-02$
```

    2. Run
       sudo docker build -t backend:1.0 .

```
azureuser@my-server-01:~/dockerfiles/my-next-backend-02$ sudo docker build -t backend:1.0 .
[+] Building 2.8s (1/2)
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 381B
 => [internal] load metadata for docker.io/library/node:20-alpine
```

2. Once completed, check your image output with command:
   sudo docker image ls

```
azureuser@my-server-01:~/dockerfiles/my-next-backend-02$ sudo docker image ls

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
backend:1.0    9b62ec2f9c9f        983MB          213MB
azureuser@my-server-01:~/dockerfiles/my-next-backend-02$
```

   Observe if your image tag (with option "-t") appear in the list.

3. Now, change to the frontend directory

```
azureuser@my-server-01:~/dockerfiles/my-next-backend-02$ cd ..
azureuser@my-server-01:~/dockerfiles$ cd my-react-frontend/
azureuser@my-server-01:~/dockerfiles/my-react-frontend$
```

4. Run the similar command with different image name

   sudo docker image build -t frontend:1.0 .

```
azureuser@my-server-01:~/dockerfiles/my-react-frontend$ sudo docker image build -t frontend:1.0 .
[+] Building 2.3s (2/3)
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 343B
 => [internal] load metadata for docker.io/library/node:20-alpine
 => [internal] load metadata for docker.io/library/nginx:alpine
```

5. Once completed, also observe if the image is on the docker image list.

```
azureuser@my-server-01:~/dockerfiles/my-react-frontend$ sudo docker image ls

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
backend:1.0    9b62ec2f9c9f        983MB          213MB
frontend:1.0   6526056eb9a9         93MB         26.2MB
azureuser@my-server-01:~/dockerfiles/my-react-frontend$
```

6. Go back to the "dockerfiles" directory.
7. Grab the file "docker-compose.yml" from MS Team.
8. Using "vi" or "nano" to modify:
    1. CERTBOT_EMAIL
    2. Image name and version for frontend and backend which matched to what you built
    3. Your Mongo DB authentication as you specified in ".env.local"
    4. JWT Secret key
9. Grab the file "nginx-certbot.env" from MS Team and saved into the "dockerfiles" directory.
10. Modify CERTBOT_EMAIL so it matched to what you defined in step 8.
11. Make a directory named "user_conf.d"
12. Change to the "user_conf.d" directory.
13. Create and edit a new file with nano or vi and named title file to match your Azure DNS with ".conf", for example
    nano athiphathrn-server-01.southeastasia.cloudapp.azure.com.conf

```
azureuser@my-server-01:~/dockerfiles/my-react-frontend$ cd ..
azureuser@my-server-01:~/dockerfiles$ nano docker-compose.yml
azureuser@my-server-01:~/dockerfiles$ mkdir user_conf.d
azureuser@my-server-01:~/dockerfiles$ cd user_conf.d
azureuser@my-server-01:~/dockerfiles/user_conf.d$ nano athiphathrn-server-01.southeastasia.cloudapp.azure.com.conf
```

14. Edit the content as shown below and change the server's name to match your server's name.

```
server {

    # Listen to port 443 on both IPv4 and IPv6.

    listen 443 ssl;



    # Domain names this server should respond to.

    server_name athiphathrn-server-01.southeastasia.cloudapp.azure.com;



    # Load the certificate files.

    ssl_certificate         /etc/letsencrypt/live/athiphathrn-server-
01.southeastasia.cloudapp.azure.com/fullchain.pem;

    ssl_certificate_key     /etc/letsencrypt/live/athiphathrn-server-
01.southeastasia.cloudapp.azure.com/privkey.pem;
```

```
    ssl_trusted_certificate /etc/letsencrypt/live/athiphathrn-server-
01.southeastasia.cloudapp.azure.com/chain.pem;


    # Load the Diffie-Hellman parameter.
    ssl_dhparam /etc/letsencrypt/dhparams/dhparam.pem;


    location /profile-images {
        alias /var/www/html/public/profile-images;
        try_files $uri =404;
         add_header Cache-Control "no-store";
    }


    location /api {
        proxy_pass http://backend-server:3000/api;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_http_version 1.1;
        proxy_cache_bypass $http_upgrade;
    }


    location / {
        proxy_pass http://frontend-server:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_http_version 1.1;
        proxy_cache_bypass $http_upgrade;
```

```
    }

}
```

15. Try to start it one by one to see if there is any problem
    1. Start the backend:
       sudo docker compose up -d my-next-backend

```
azureuser@my-server-01:~/dockerfiles$ sudo docker compose up -d my-next-backend
WARN[0000] No services to build
[+] up 3/3
 ✔ Network dockerfiles_default                 Created
 ✔ Volume dockerfiles_next_user_profile_images Created
 ✔ Container backend-server                    Created
azureuser@my-server-01:~/dockerfiles$
```

       Checking the running container with:
       sudo docker ps -a

```
azureuser@my-server-01:~/dockerfiles$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED             STATUS             PORTS                                          NAMES
2b726a7956f7   backend:1.0   "docker-entrypoint.s…"   About a minute ago  Up About a minute  0.0.0.0:32768->3000/tcp, [::]:32768->3000/tcp  backend-server
```

       Check the server logs using:
       sudo docker compose logs my-next-backend

```
azureuser@my-server-01:~/dockerfiles$ sudo docker compose logs my-next-backend
backend-server  |
backend-server  | > my-next-backend-02@0.1.0 start
backend-server  | > next start
backend-server  |
backend-server  | ▲ Next.js 16.1.1
backend-server  | - Local:         http://localhost:3000
backend-server  | - Network:       http://172.18.0.2:3000
backend-server  |
backend-server  | ✓ Starting...
backend-server  | ✓ Ready in 305ms
```

    2. Doing the same step for my-react-frontend
    3. And lastly for nginx
16. Once you start all containers, you may observe if nginx start up properly, using

    sudo docker compose logs -f ngin

    Observe if any error is shown.

17. To check if SSL can be setup properly or not, you need to grant yourself as “root” permission and check the files in docker volume:
    1. Grant yourself as root:
       sudo su

    2. Change directory to
       /var/lib/docker/volumes/

    3. You shall see your directories related to the one you set up in docker compose:

```
root@my-server-01:/var/lib/docker/volumes# ls -al
total 48
drwx-----x  5 root root  4096 Jan 13 06:31 .
drwx--x--- 11 root root  4096 Jan 13 06:19 ..
brw-------  1 root root  8, 1 Jan 13 05:40 backingFsBlockDev
drwx-----x  3 root root  4096 Jan 13 06:19 dockerfiles_next_user_profile_images
drwx-----x  3 root root  4096 Jan 13 06:31 dockerfiles_nginx_secrets
drwx-----x  3 root root  4096 Jan 13 06:27 dockerfiles_public_react
-rw-------  1 root root 32768 Jan 13 06:31 metadata.db
```

    4. Change directory to
       cd dockerfiles_nginx_secrets/_data

    5. Listing if any files and directories are here

```
root@my-server-01:/var/lib/docker/volumes/dockerfiles_nginx_secrets/_data# ls -l
total 24
drwx------ 3 root root 4096 Jan 13 06:31 accounts
drwx------ 3 root root 4096 Jan 13 06:49 archive
drwxr-xr-x 2 root root 4096 Jan 13 06:31 dhparams
drwx------ 3 root root 4096 Jan 13 06:49 live
drwxr-xr-x 2 root root 4096 Jan 13 06:49 renewal
drwxr-xr-x 5 root root 4096 Jan 13 06:31 renewal-hooks
```

18. Don’t forget to add your Azure server’s public IP address to your Cloud Mongo DB or your backend will be unable to connect.
19. Access your application at “https://<your-azure-dns>”

Trouble Shooting

- Check your docker-compose.yml to ensure the Mongo DB URI.
- Check Mongo DB available IP.
- Check the nginx configuration to ensure that the server’s name is correct and matches your Azure DNS.
- Recheck if you apply the downloaded configuration to the correct directory.

Docker common command

To list the running container:

```
docker ps
```

To list all containers:

```
docker ps -a
```

To remove the container data:

```
docker container prune
```

To build an image

```
docker image build -t <image name>:<version> .
```

To list images

```
docker image ls
```

Docker-compose command

Note that to run the docker-compose you must be at the same directory of a “docker-compose.yml”

To start a specific container, first time

```
docker compose up -d <compose-config-name>
```

To start all containers at once

```
docker compose up -d
```

To resume containers (from stop)

```
docker compose start <compose-config-name>
```

To stop a container

```
docker compose stop <compose-config-name>
```

Common step to restart the container with new image built:

- Update built version in docker-compose.yml
- Stop the container (compose stop)
- Prune the container (container prune)

- Run the container (compose up -d)

Be careful that every data that is not stored in explicit volume will be permanently removed with prune command!!